HPC Project Report

Leonardo Antonio Riola, Davide Donati, Giacomo Girotto

1 Assignment 1

1.1 Introduction and objectives

The objective of this assignment is to implement a simulator that models the propagation of a damped wave. The general partial differential equation describing the phenomenon in space as time varies is the following:

$$\frac{\partial^2 u}{\partial t^2} + \gamma \frac{\partial u}{\partial t} = c^2 \nabla^2 u \tag{1}$$

Where $u(x, y, t)$ represents the height of the wave on a two-dimensional plane over time, γ is the wave damping coefficient, and c is the wave propagation speed. The simulator calculates the wave height for each point in the coordinate space (x, y) as time t changes, with the ultimate goal of generating an MP4 video of the simulation.

1.2 Implementation

To perform the simulation, it is necessary to convert the continuous model into a discrete model suitable for implementation on a computer. For this purpose, a grid is introduced: space is divided into equally spaced points along the x and y directions with steps Δx and Δy, and time is discretised into instants separated by a time step Δt. Consequently, each point in space is identified by integer indices (i, j), and time by an index t. The continuous function $u(x, y, t)$ is replaced by a matrix of values $u^t_{i,j}$, representing the value of the function at the grid point (i, j) at time step t. The derivatives appearing in the differential equation are approximated using finite differences. In particular, both **central and forward difference** methods are implemented in the code and can be tested separately.

To handle the temporal evolution of the wave, three distinct $M \times M$ matrices are dynamically allocated in memory using the `malloc` function in C. These matrices, defined as `u_past`, `u`, and `u_fut`, represent the wave state at the previous time step $(t - 1)$, the current time step (t), and the future time step $(t + 1)$, respectively. Initially, all matrix elements are set to zero but an initial impulse in a specific locationis assigned to start the perturbation. In the analyzed case, the initial impulse, which is set at the center of the grid (M/2, M/2), the propagation speed c and the damping coefficient γ are set to the following values:

Impulse	**Propagation speed** c	**Damping coefficient** γ
-49	0.33	0.187

To accelerate the computation, the program utilizes the **OpenMP** library as parallelization framework to generate the matrices representing the evolution of the wave, which is also the core of this assignment. Instead of calculating the new state of the grid sequentially, the workload is distributed across multiple active threads: this is performed using the `#pragma omp parallel` directive within the code just before the loop for the temporal advancement. Then, by parallelizing the nested spatial loops using the `#pragma omp for schedule(static) collapse(2)` directive, the computation of individual matrix elements is executed concurrently. The `schedule(static)` clause dictates that the loop iterations are divided into chunks of equal size and assigned to the threads before execution begins. The `collapse(2)` clause is utilized to fuse the two nested spatial loops (iterating over rows i and columns j) into a single iteration space of size $M \times M$. This significantly increases the total number of independent iterations available for distribution, ensuring a finer and more balanced division of the workload among the active cores. By doing so, while one thread computes a future value $u^{t+1}_{i,j}$

for a specific coordinate utilizing the current and past values ($u_{i,j}^{t}$ and $u_{i,j}^{t-1}$), all the other threads are simultaneously computing the future values for different coordinates.

Once the entire `u_fut` matrix has been fully calculated for the current temporal iteration, the program must write the costructed matrix to a Portable Graymap (PGM) file format. To do so, the program uses a `#pragma omp single` directive to ensure that only one thread performs the writing operation, while the other threads will wait at an implicit synchronization barrier. Finally, the simulation must advance to the next time step. To maximize efficiency and eliminate the computational overhead of deep-copying large memory blocks, time progression is achieved through a cyclic pointer swap (`u_past` takes the address of `u`, `u` points to `u_fut`, and `u_fut` is recycled to store the next iteration). This operation is executed by the same thread which performed the writing to the PGM file, guaranteeing that only one designated thread performs the pointer exchange, preventing race conditions and strictly preserves memory efficiency before the threads begin computing the next time step.

1.2.1 Identified inefficiencies and corrective transformations

All profiling and testing in this section was performed using 32 threads with a workload size of $M = 1000$.

The initial implementation, although parallelized with OpenMP, was profiled using **Intel VTune Profiler**, combining *Hotspots* analysis (which functions consume CPU time) and *Threading* analysis (how much of that time is effective computation versus idle spinning at synchronization points).

Hotspot analysis showed that `gomp_team_barrier_wait_end` and `fwrite` accounted for 79.4% and 9.7% of total CPU time respectively, against only 2.2% for the matrix computation itself. The bottleneck was therefore not the stencil, but excessive synchronization and I/O overhead. Three successive transformations addressed this, each verified via VTune or direct code inspection.

Improvement 1: buffered frame output. Frames were originally written pixel-by-pixel with `fwrite()` inside a single-threaded `#pragma omp single` block. This was replaced with an in-memory buffer, filled in parallel via a dedicated `#pragma omp for` and flushed to disk with one `fwrite()` call, cutting I/O calls from one per pixel to one per frame. This introduced a new barrier, since the buffer-fill loop is a separate worksharing construct from the stencil loop.

Improvement 2: loop fusion. As the stencil and buffer-fill share the same (i, j) range, they were merged into a single `#pragma omp for`, writing each scaled pixel immediately after computing `u_fut[i][j]`. This removes the extra barrier from Improvement 1 with no loss of parallel work.

Improvement 3: NUMA-aware initialization. On a multi-socket machine, each CPU socket has its own memory: cores access memory local to their own socket faster than memory attached to another socket (NUMA). Physical pages are assigned to a NUMA domain on *first touch* (the first thread to write to a page). A serial initialization therefore places all matrix memory on a single NUMA domain; once the parallel calculation spreads work across all cores, threads mapped to a different socket pay a remote-access penalty on every timestep. To prevent this, the initialization loop was parallelized with the same `schedule(static) collapse(2)` clause used in the calculation loop, so that each thread's first-touch region matches the region it repeatedly accesses afterwards. This requires a *static* schedule specifically: a dynamic schedule would not guarantee that the same thread is assigned the same iterations across the two loops, making the first-touch alignment ineffective.

After these improvements, from the VTune Hotspot analysis, the CPU time spent in `gomp_team_barrier_wait_end` dropped to 57.5%, while the time spent in matrix computation rose to 15.7%. The I/O (`fwrite` CPU time) time instead dropped significantly to less than 0.3%. However, inspecting the Threading analysis, it was found that the program still used only 23% of the available CPUs, resulting in an average effective CPU utilization of only 7.362 out of 32. This was due to the too low workload size compared to the number of threads. The next steps inspected and tested the program changing the workload size and other parameters to see how the program utilizes the resources differently.

1.3 Test

1.3.1 Sweep of workload size

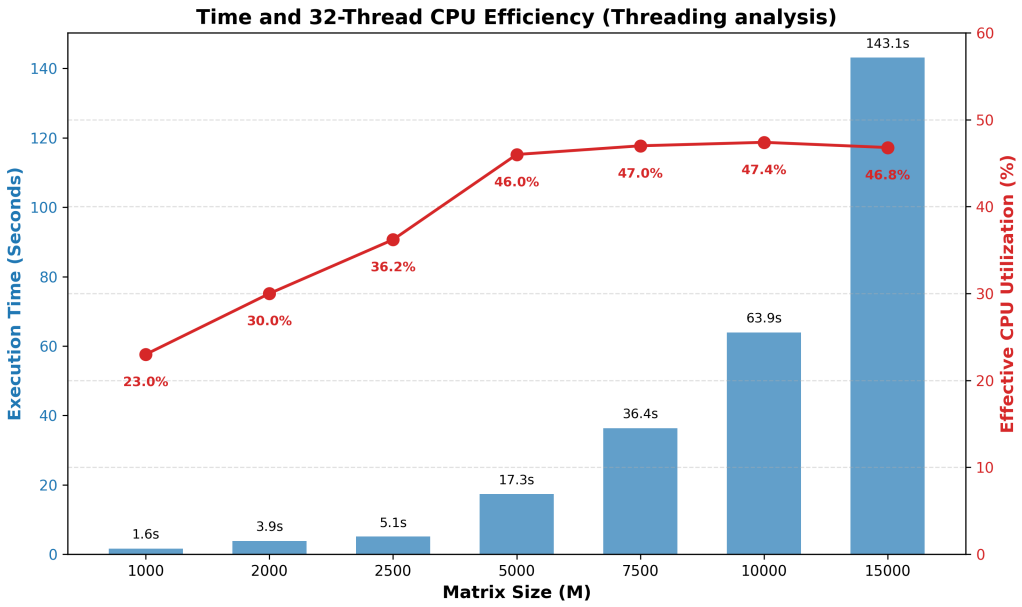

Figure 1: Execution time and True CPU Efficiency scaling with varying grid sizes (M) using 32 threads.

1.3.2 I/O test

run the program with and without the write amtrix function to see the difference.

1.3.3 Sweep of number of threads per matrix dimension

in order to see how scale up the efficiency of parallelization (also commenting the I/O part to see only the computation time)

2 Assignment 2

2.1 Introduction

Following the initial wave simulator developed with OpenMP, this second assignment extends the application to execute multiple concurrent simulations on an HPC system using a hybrid MPI and OpenMP programming model. The goal is to study multi-wave interaction and interference dynamics under different physical conditions. While OpenMP continues to handle data parallelism by accelerating the 2D stencil computations across local CPU cores, MPI is introduced to manage task parallelism. Specifically, the system executes three distinct simulation scenarios simultaneously within a single execution run:

- Simulation 1: Reproduces the baseline single-wave scenario initiated by a single central impulse at $t = 0$.
- Simulation 2: Models wave superposition and interference phenomena by initiating two simultaneous impulses at different spatial locations at $t = 0$.
- Simulation 3: Introduces time-delayed interference dynamics by generating a second impulse at a spatial location after a predetermined time-shift ($t = t_{start}$).By decoupling these independent workloads across separate MPI processes, the hybrid architecture maximizes computational throughput without requiring inter-process communication during the time-stepping loop.

By decoupling these independent workloads across separate MPI processes, the hybrid architecture maximizes computational throughput without requiring inter-process communication during the time-stepping loop.

2.2 Implementation

Building upon the code developed in Assignment 1, the implementation extends the simulator to handle multiple concurrent runs. The core numerical scheme for the time-stepping propagation and the mapping function converting field values into grayscale images (**save_pgm**) remain identical to the first version, ensuring consistency in wave physics and visualization. The execution starts by initializing the MPI environment with **MPI_Init**, retrieving the total number of processes with **MPI_Comm_size**, and identifying the rank ID of each process with **MPI_Comm_rank**. To run the three required simulations simultaneously, the program verifies that at least 3 processes are allocated. Task parallelism is managed through conditional branching based on the process rank, where each rank executes a specific scenario and saves its frames into a dedicated directory:

- **Rank 0** (*sim1*): Handles the baseline single-wave simulation initiated by a central impulse of **-84.0f** at coordinates $(M/2, M/2) = (200, 200)$ at $t = 0$.
- **Rank 1** (*sim2*): Manages the simultaneous two-wave interference scenario, applying the central impulse **-84.0f** at $(200, 200)$ alongside a second impulse of **36.0f** at coordinates $(3M/4, M/3) = (300, 133)$ at $t = 0$.
- **Rank 2** (*sim3*): Manages the time-delayed scenario, applying the central impulse **-84.0f** at $(200, 200)$ at $t = 0$ and scheduling a second impulse of **60.0f** at coordinates $(3M/4, 2M/3) = (300, 266)$ to be injected later at time step $n_{\text{start}} = N/7$.

Each MPI process independently allocates memory using **malloc** for its local grid arrays (**u_prev**, **u_curr**, and **u_next**). While data parallelism within each process is still handled by OpenMP as established in Assignment 1, the time-stepping loop incorporates rank-specific logic. Specifically, the check for **rank == 2** at **n == n_start** is placed inside a **pragma omp single** directive within the time loop. This ensures that the time-delayed second impulse for Simulation 3 is injected into the grid at the exact required time step by only one thread per process, avoiding race conditions and preventing multiple threads from applying the impulse redundantly. The same **pragma omp single** block also safely manages frame saving with **save_pgm** and array pointer swapping without thread interference.

The job execution on the HPC cluster is controlled via a SLURM batch script. The configuration requests 3 MPI tasks (*ntasks=3*) to match the 3 simulation ranks, and reserves 4 CPU cores per task (*cpus-per-task=4*). By setting **export OMP_NUM_THREADS** dynamically to the allocated CPUs per task, each MPI process spawns four OpenMP threads, effectively utilizing a total of 12 CPU cores in parallel across the compute node.

2.3 Test and optimizaion

3 Assignment 3